5 Hacks to Build Professional Websites with Claude Code

This guide covers five practical hacks for using Claude Code to build websites that look professional and branded, not like they were AI vibe-coded. Even if you've never used Claude Code before, you'll be able to follow along and spin up polished landing pages.

By: Nate Herk

Getting Started: Setup

Before diving into the hacks, you need three things in place:

1. **Download Visual Studio Code (VS Code)**: Go to your browser, search "VS Code," and download it for your operating system. This is the IDE (code editor) where you'll use Claude Code.

2. **Install the Claude Code Extension**: Open VS Code, click on **Extensions** in the left sidebar, search for "Claude Code," and install it. You'll be prompted to sign in with your Anthropic/Claude subscription (a paid account is required, Pro or Max).

3. **Open a Project Folder**: Click on **Explorer** in the top-left of VS Code and open (or create) an empty folder. This is where all your website files will live. The left side of VS Code shows your files/folders, and the right side is where you interact with the Claude Code agent.

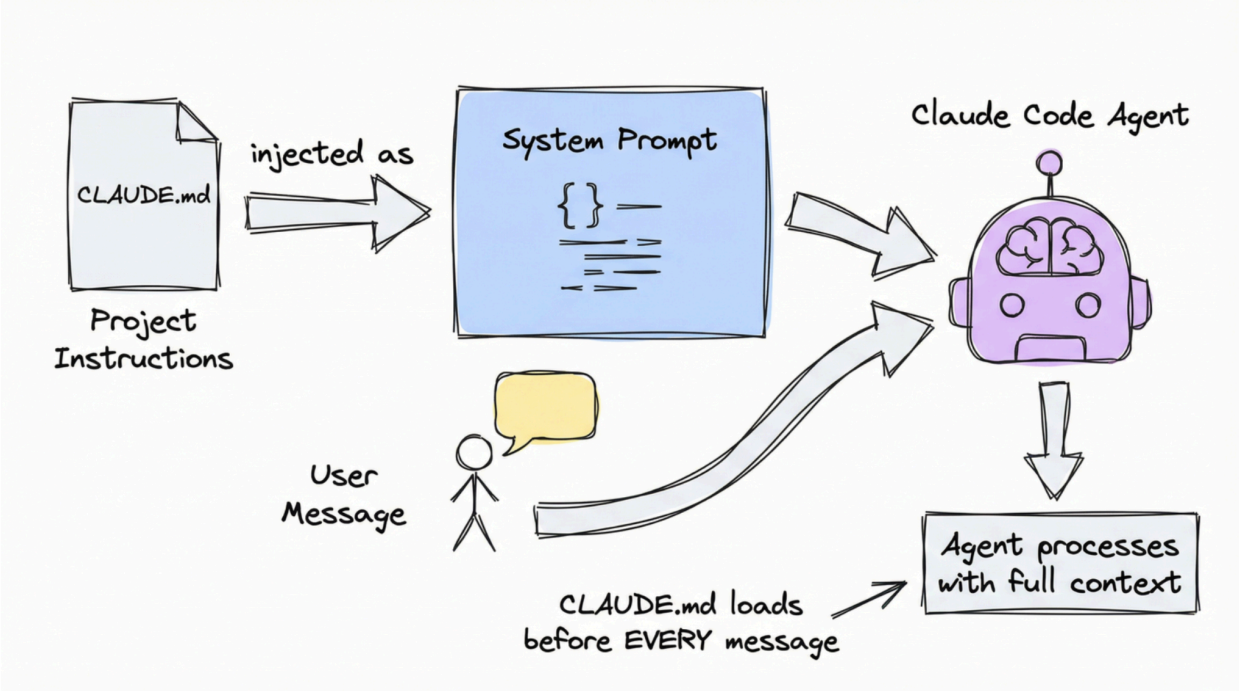
Hack #0: The CLAUDE.md File

Think of the `CLAUDE.md` file as a **system prompt** for Claude Code. Every time you ask Claude Code to do something, it reads this file first. It sets the rules, goals, and guidelines for the entire project.

Key points about your CLAUDE.md:

- Keep it concise, don't bloat it with too much context.
- Define the rules you want Claude Code to follow every time it works on your project.

- You may not know your full process at the start, and that's okay, you can iterate on it throughout the project.
- A free web design CLAUDE.md file is available in the free Skool community under Classroom → Claude Code.



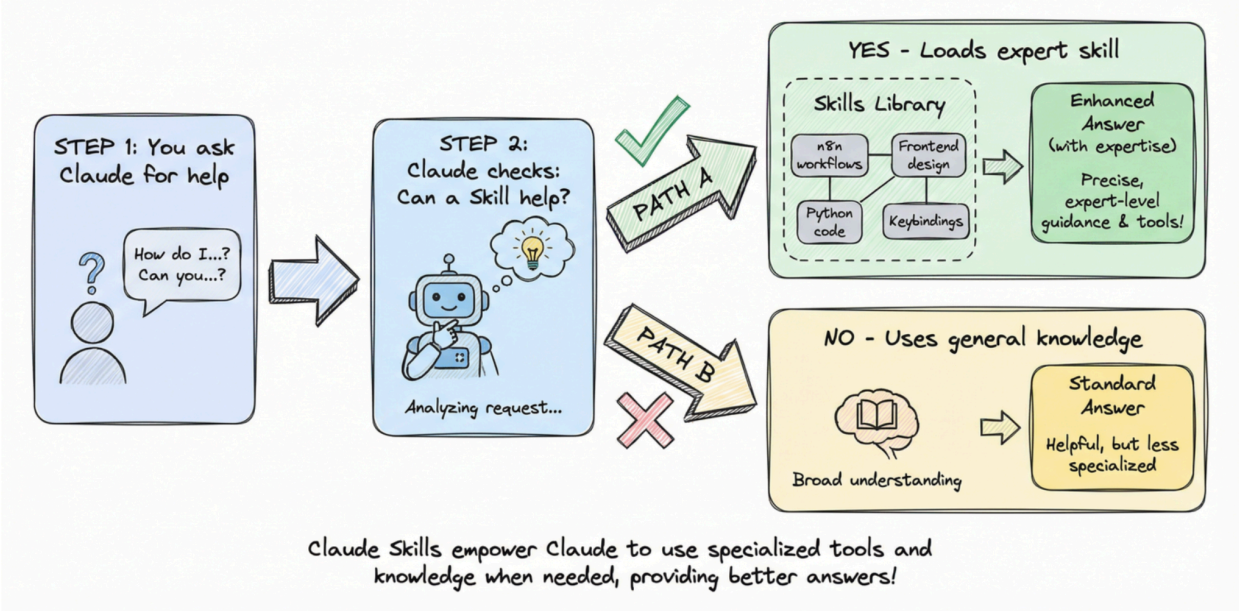
How to add it: Simply drag the `CLAUDE.md` file into your project folder on the left side of VS Code. The `.md` stands for Markdown, which uses pound signs for headers, asterisks for bold, and bullet points to keep the text organized for the agent.

> **Note:** The CLAUDE.md is listed as Hack #0 because it's a prerequisite, but you'll also keep coming back to update and refine it as your project evolves.

Hack #1: The Front-End Design Skill

Skills are **custom instruction sets** stored as markdown files. When you ask Claude Code a question, it first reads the CLAUDE.md, then checks if any installed skills are relevant to your request. If a matching skill exists, Claude reads it and uses that specialized knowledge. If not, it falls back to its general knowledge.

The Front-End Design Skill specifically helps Claude Code create designs that are modern, professional, and don't look AI-generated. Without it, you might get a basic-looking site. With it, you get animations, dynamic elements, and a much more polished result.



How to install it: The skill is installed by running two commands (linked in the original video description via this tweet). You can copy and paste those commands directly into Claude Code. Once installed, the skill works **globally** across all future Claude Code projects.

In your CLAUDE.md, add a line like: *"Always invoke the front-end design skill before writing any front-end code every session, no exceptions."*

Demo result: With just a one-sentence prompt, "Build me a modern and professional landing page for AI Automation Society", plus the front-end design skill and brand assets, Claude Code produced a full one-page landing page with animations, scrolling tech company logos, testimonials, a call to action, a floating logo effect, and correct brand colors/typography.

Bonus Hack: Brand Assets Folder

Create a folder in your project called `brand_assets` and add two things:

1. **Your logo** (image file)
2. **Brand guidelines** (a document with your colors, typography, icons, etc.)

Your CLAUDE.md should reference this folder so Claude Code knows to look at it. You can also **tag assets directly** in your prompt using the `@` symbol to explicitly point Claude Code to the right files, which is especially useful if your file names aren't super intuitive.

Hack #2: The Screenshot Loop

AI is good at getting you most of the way, but it takes manual correction and steering to close the gap. The screenshot loop lets Claude Code **iterate on itself** by taking screenshots of what it's building, comparing them to the target, and making improvements automatically.

How it works:

- Claude Code uses **Puppeteer** (a browser automation tool) to take screenshots of the website it's building.
- It then analyzes the screenshots section by section to check how well the output matches what's expected.
- It performs multiple passes (typically two rounds of comparison), fixing mismatches each time.
- Screenshots are saved in a `temporary_screenshots` folder in your project.

Setup: In your CLAUDE.md, include a section for the screenshot workflow. You can literally tell Claude Code: "Hey, can you set up Puppeteer to take screenshots?" and it will install everything needed.

Pro Tip: The screenshots are primarily for Claude Code's benefit, not yours. However, if you want to click through and see changes between versions, be more specific about naming conventions in your CLAUDE.md. Also, remember to periodically tell Claude Code to delete old temporary screenshots to keep things clean.

Hack #3: Using Other Websites as Inspiration

You can give Claude Code a reference website and ask it to **clone the layout and style**. Combined with the screenshot loop, it will build, compare against the reference, and iterate until it's close.

Great sources for website inspiration:

- Dribbble
- Godly Website
- Awwwards

Step-by-step process to clone a website:

1. **Capture a full-page screenshot** of the reference site, Hit `F12` to open DevTools, then `Ctrl+Shift+P` (Windows) and search for "screenshot" → "Capture full-size

screenshot." This downloads an image of the entire page, not just the current viewport.

2. **Copy the style information**, In DevTools, go to **Elements**, then in the **Styles** section, copy everything. This gives Claude Code the raw styling/CSS information.

3. **Prompt Claude Code**, Paste the style code, drag in the screenshot, and tag it. Example prompt: *"I want you to spin up a new website. Clone this website. Here's a screenshot and here's the style."*

4. **Claude Code builds, screenshots, and compares**, It will write the code, start a server, take screenshots of its work, compare them to your reference, and do multiple rounds of fixes.

5. **Apply your branding**, Once the clone looks good, tell Claude Code to work in your brand assets (logo, colors, brand guidelines). Example: *"The most recent landing page looks really good. Work in our brand assets, our brand guidelines and AIS logo. This is for our community called AI Automation Society."*

Important note on Bypass Permissions Mode: If Claude Code keeps stopping to ask for permissions, go to **Settings → search "Claude Code" → Allow Dangerously Skip Permissions**. This lets it run commands without pausing for approval. Use with caution, don't leave it running unattended overnight, but in practice it's generally fine when you're nearby monitoring.

Hack #4: Individual Components

Once you have a website you like the overall feel of, you can level it up by pulling in **individual components** from inspiration sources rather than cloning entire websites.

Recommended resource: 21st.dev, a library of high-quality website components including shaders, backgrounds, buttons, mouse highlights, hero elements, and more. Each component has a dark/light mode toggle and a copy prompt button.

How to use it:

1. Browse 21st.dev and find a component you want (e.g., a hero wave background, a glowing button, a special animation).
2. Click "Copy Prompt" on the component to grab the code.
3. Tell Claude Code where to place it. Example: *"I want you to work in this background element right behind the hero text"*, then paste in the copied code.

When to disable the screenshot loop: For animated or dynamic components, screenshots may not capture the animation properly. This can cause Claude Code to get stuck in a loop thinking its output isn't good enough. In these cases, add to your prompt: *"Because this is an animated background, do not use the screenshot tool to compare. Just work in the code and I will let you know if we need to make any changes."*

Iterating with feedback: After Claude applies a component, give specific verbal feedback on what to improve. For example: *"The background animation is too distracting and pixelated. The hero text is hard to read. Add a background behind the hero text for contrast. Change 'earn more' to blue instead of orange."* Claude Code will interpret your creative direction and make the adjustments.

Deploying Your Website: GitHub + Vercel Pipeline

Once your site looks good on localhost, here's how to get it onto a real live domain.

The flow: Claude Code (local files) → GitHub (cloud-hosted code with version control) → Vercel (live deployed website)

Step 1: Push to GitHub

1. Go to GitHub and create a new repository (e.g., "AIS-test-website").
2. In Claude Code, tell it: *"This site looks good. Help me push this to GitHub to a repository called [repo name]."*
3. Authenticate with your GitHub credentials when prompted.
4. Claude Code will create a `.gitignore`, set everything up, and push your code.

Security reminder: Before pushing to GitHub, make sure your project doesn't contain API keys, passwords, usernames, webhook URLs, or any sensitive information.

Step 2: Connect to Vercel

1. Go to Vercel and create an account (easiest if you sign up with your GitHub credentials).
2. Click **Add New Project** → choose your GitHub repository → click **Import** → click **Deploy**.
3. Your site is now live at a Vercel URL (e.g., `ais-test-website.vercel.app`).

Step 3: Add a Custom Domain

1. In Vercel, go to **Project Settings** → **Domains**.
2. Buy a new domain or add an existing one.
3. Follow the DNS configuration walkthrough that Vercel provides.

The Auto-Deploy Workflow

The real power of this setup is the **auto-deploy pipeline**:

1. Make changes locally with Claude Code and preview on localhost.
2. When you're happy, tell Claude Code: *"Push that to GitHub."*
3. GitHub receives the new commit → Vercel automatically detects it → the live site updates.

Important CLAUDE.md addition: Add instructions like: *"When making changes, always test on localhost first. Don't commit or push to GitHub until I explicitly tell you to."* This prevents untested changes from going live.

Example workflow:

- You ask Claude Code to add a glowing effect to a button.
- It makes the change and you preview it on localhost.
- You like it → *"Awesome, push that to GitHub."*
- Within moments, the live Vercel site reflects the update.

Want to connect with others building and monetizing AI automation?

Become an AIS Plus Member